EduGenie

Google Gemini Powered Learning Assistant

Detailed Project Report

Technology Track: Generative AI with Google

Project Type: Generative AI Learning Assistant

Prepared for Academic Project Submission

Executive Summary

EduGenie is a Generative AI powered learning assistant designed to help students understand academic concepts in a simpler, interactive and personalized way. The system uses Google Gemini as the core language model and provides a lightweight web interface through which learners can ask questions, simplify difficult topics, generate quizzes, obtain summaries and receive study recommendations.

The project focuses on reducing the difficulty students face when learning from complex notes, textbooks and technical explanations. Instead of requiring a learner to formulate a perfect prompt, EduGenie organizes common learning tasks into dedicated functions. A student can enter a topic or question, choose the required learning action, and receive an AI-generated response in an easy-to-read format.

The proposed implementation uses a Python FastAPI backend and a simple HTML/CSS frontend. The backend receives requests from the browser, prepares a structured prompt, sends it to the Gemini model through the Google Generative AI interface, and returns the generated result to the user interface. API credentials are kept outside source code through an environment variable.

Core capabilities include question answering, concept simplification, quiz generation, study recommendations and summarization. The design can be extended with user profiles, learning history, difficulty adaptation and additional educational tools.

Core Value Proposition: Convert a student's question or learning material into clear explanations, summaries, quizzes and actionable study guidance through one simple AI-assisted interface.

1. Introduction

1.1 Background

Generative AI has created new ways for learners to interact with information. Traditional digital learning resources generally provide fixed explanations, while a conversational AI system can adapt an explanation to the learner's question. EduGenie uses this capability as an academic support tool, with the goal of making difficult concepts easier to understand and revise.

1.2 Problem Statement

Students commonly face difficulty understanding complex technical concepts, lengthy study material and unfamiliar terminology. Searching across multiple websites can also take time, and a learner may receive information that is too advanced or not organized for revision. There is therefore a need for a simple assistant that can provide explanations, summaries and practice material in response to a student's specific request.

1.3 Proposed Solution

EduGenie provides a unified learning interface connected to Google Gemini. The application accepts a learning request, identifies the selected task, constructs an appropriate prompt, obtains an AI response and presents the result in a student-friendly format. The system is designed as an educational assistance tool rather than a replacement for teachers, textbooks or institutional learning resources.

2. User and Use-Case Analysis

2.1 Target Users

User Group	Need	EduGenie Support
School / college students	Understand difficult concepts and revise	Explanations, summaries and quizzes
Computer science learners	Understand programming and technical topics	Simplification and Q&A;
Exam preparation learners	Practice questions and quick revision	Quiz generation and summaries
Self-learners	Get guided explanations without switching tools	Conversational learning assistance
Beginner learners	Avoid overly technical explanations	Simple-language mode

2.2 Main Use Cases

Use Case	Input	Output
Ask a Question	Topic/question	Direct AI explanation
Simplify Concept	Complex concept or paragraph	Simple explanation with examples
Generate Quiz	Topic + number/difficulty	Questions with answers
Summarize	Study text	Short structured summary
Study Recommendation	Topic + learner need	Suggested learning sequence/resources

2.3 Functional Requirements

• The application shall accept a learner's text input.
• The application shall allow selection of a learning task.
• The backend shall construct task-specific prompts.
• The system shall send requests to the Gemini model.
• The generated response shall be returned to the frontend.
• Quiz mode shall generate questions and answers in a structured format.
• The application shall handle missing input and API errors gracefully.
• API credentials shall not be hard-coded in public source files.

2.4 Non-Functional Requirements

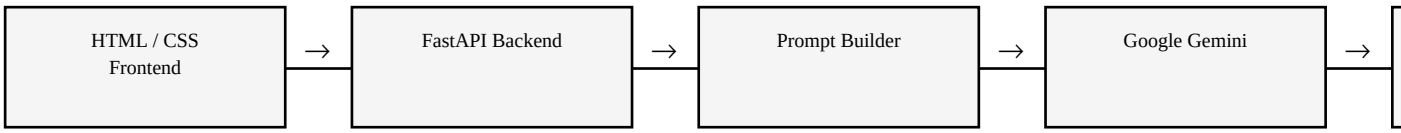
Usability: clear controls and readable output. Performance: avoid unnecessary processing and return responses efficiently. Security: protect the Gemini API key through environment configuration. Maintainability: separate frontend, API routes and Gemini integration. Scalability: keep the backend modular so additional learning modules can be added later.

3. Technical Architecture

3.1 System Overview

EduGenie follows a simple layered architecture. The presentation layer provides the student interface, the API layer manages requests, the AI service layer communicates with Gemini, and the response layer formats the generated content for display.

EduGenie Request Flow



3.2 Core Components

Component	Purpose
Frontend	Collects learning input and displays results.
FastAPI Backend	Provides HTTP endpoints and validates requests.
Prompt Builder	Creates task-specific instructions for Gemini.
Gemini Integration	Sends the prompt to Google's Gemini model.
Response Formatter	Organizes generated content for the UI.
Environment Configuration	Stores API credentials outside source code.

3.3 Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User interface and API calls
Backend	Python + FastAPI	REST API and request handling
AI	Google Gemini	Generative AI response generation
Configuration	.env / environment variables	API key management
Development	VS Code / Python 3.10+	Implementation and testing
Version Control	Git / GitHub	Source-code management and project phases

3.4 Data Flow

1. Student opens EduGenie. 2. Student selects a learning task. 3. Student enters a topic, question or study text. 4. Frontend sends an HTTP request to FastAPI. 5. Backend validates the input and constructs a prompt. 6. Prompt is sent to Gemini. 7. Gemini generates a response. 8. Backend returns the response as JSON. 9. Frontend renders the learning result.

4. Functional Modules

4.1 Question Answering Module

The Question Answering module accepts a learner's question and instructs Gemini to provide a direct educational answer. The prompt can request an appropriate level of detail and examples so the output is more useful for learning rather than simply returning a short factual response.

4.2 Concept Simplifier

The simplifier converts complex terminology or an academic paragraph into simpler language. A useful response structure is: simple definition → explanation → example → key points. This helps beginners approach topics that may initially appear difficult.

4.3 Quiz Generator

The quiz module asks Gemini to create practice questions from a specified topic. The prompt can define the number of questions, difficulty and whether answers or explanations should be included. The resulting quiz can be used for self-assessment.

4.4 Summarizer

The summarizer accepts study material and asks Gemini to produce a concise structured summary. It can organize the output using headings, bullet points, important terms and key takeaways.

4.5 Recommendation Module

The recommendation module provides a learning sequence based on a topic. Rather than acting as a formal academic advisor, it provides general study guidance such as prerequisite concepts, suggested order of learning and practice activities.

4.6 Error Handling

The backend should return understandable errors for empty input, invalid requests, unavailable API responses and configuration problems. The frontend should display a clear message instead of exposing internal exception details or secret credentials.

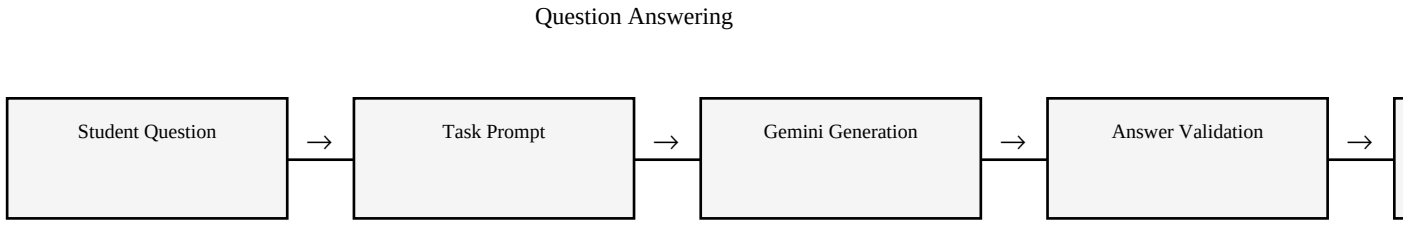
5. AI Prompt and Workflow Design

5.1 Prompt Strategy

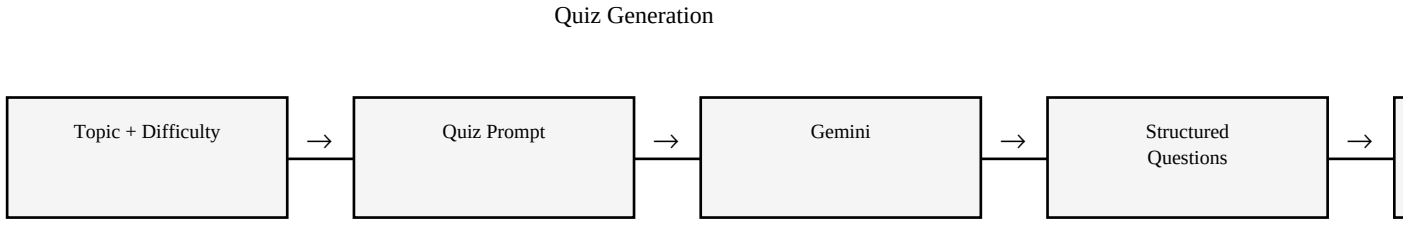
EduGenie uses task-specific prompts instead of sending raw user input directly to the model. The prompt identifies the role of the model, the requested educational task, the learner's input and the desired output format. This improves consistency and makes the application easier to maintain.

```
SYSTEM / ROLE
You are EduGenie, an educational learning assistant.
Explain concepts clearly and use examples when useful.

TASK
Perform the selected learning action.

USER INPUT
{student_input}

OUTPUT RULES
- Use simple, student-friendly language.
- Structure the answer with headings or bullet points.
- Avoid unnecessary repetition.
- If the request is ambiguous, state the assumption clearly.
```

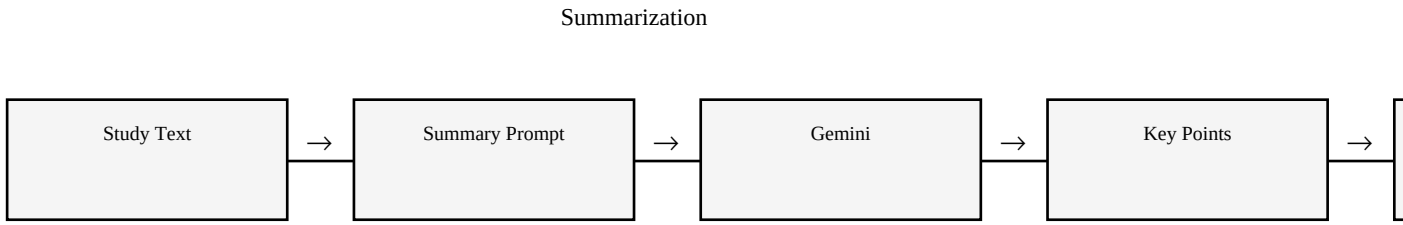
5.2 Question Answering Workflow



5.3 Quiz Workflow



5.4 Summary Workflow



5.5 Human Learning Role

AI output should be treated as learning assistance and checked against classroom notes, textbooks or other trusted sources when accuracy is important. The system should not present generated material as an official

institutional answer key unless it has been independently reviewed.

6. Implementation

6.1 Project Structure

```
edugenie/
■■■ app.py
■■■ requirements.txt
■■■ .env
■■■ README.md
■■■ static/
■   ■■■ style.css
■   ■■■ script.js
■■■ templates/
    ■■■ index.html
```

6.2 Backend Responsibilities

The FastAPI application exposes endpoints for the frontend. A simple implementation can use one general endpoint with a task parameter, or separate endpoints for Q&A;, simplification, quiz generation and summarization. The modular approach makes it easier to add features later.

6.3 Gemini Configuration

```
# .env
GEMINI_API_KEY=your_api_key_here
```

The API key should be kept in the local environment and excluded from GitHub through .gitignore. A public repository should never contain the actual API key.

6.4 Requirements

```
fastapi
uvicorn
python-dotenv
google-generativeai
jinja2
```

The exact package versions should be pinned after the team tests the project environment. The reference project materials identify Python 3.10+ and Google's Gemini integration as the intended technology direction.

7. Backend Implementation Code

7.1 FastAPI Application

```
import os
from fastapi import FastAPI
from pydantic import BaseModel
from dotenv import load_dotenv
import google.generativeai as genai

load_dotenv()

API_KEY = os.getenv("GEMINI_API_KEY")
genai.configure(api_key=API_KEY)

model = genai.GenerativeModel("gemini-1.5-flash")
app = FastAPI(title="EduGenie")

class LearningRequest(BaseModel):
    task: str
    input_text: str

@app.post("/generate")
def generate(request: LearningRequest):
    prompt = build_prompt(request.task, request.input_text)
    response = model.generate_content(prompt)
    return {"task": request.task, "result": response.text}

def build_prompt(task, text):
    return f"""
You are EduGenie, a student learning assistant.
Task: {task}
Student input: {text}

Explain clearly, use simple language,
and organize the answer for learning.
"""
```

7.2 Task-Specific Prompt Builder

```
def build_prompt(task, text):
    instructions = {
        "qa": "Answer the question clearly and give an example.",
        "simplify": "Explain the concept in beginner-friendly language.",
        "quiz": "Create practice questions with answers.",
        "summary": "Summarize the material into key points.",
        "recommend": "Suggest a logical learning sequence."
    }

    instruction = instructions.get(
        task, "Help the student understand the topic."
    )

    return f"""
You are EduGenie, an educational assistant.
Instruction: {instruction}
Student content:
{text}
"""
```

7.3 Important Implementation Note

The code above follows the technology direction of the supplied project materials and is intended as an academic reference implementation. Gemini SDK/model names can change over time; the final team environment should use the model/API configuration supported by the installed SDK and the Google account being used for the demonstration.

8. Frontend Implementation

8.1 HTML Interface

```
<!DOCTYPE html>
<html>
<head>
  <title>EduGenie - Learning Assistant</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <main class="container">
    <h1>EduGenie</h1>
    <p>Google Gemini Powered Learning Assistant</p>

    <select id="task">
      <option value="qa">Ask a Question</option>
      <option value="simplify">Simplify Concept</option>
      <option value="quiz">Generate Quiz</option>
      <option value="summary">Summarize</option>
      <option value="recommend">Study Recommendation</option>
    </select>

    <textarea id="input"
      placeholder="Enter your topic or question..."></textarea>

    <button onclick="generate()">Generate</button>
    <section id="result"></section>
  </main>
<script src="/static/script.js"></script>
</body>
</html>
```

8.2 JavaScript API Call

```
async function generate() {
  const task = document.getElementById("task").value;
  const input_text = document.getElementById("input").value;

  const response = await fetch("/generate", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({task, input_text})
  });

  const data = await response.json();
  document.getElementById("result").innerText = data.result;
}
```

8.3 Interface Design Principles

Use a single clear input area, visible task selection, a prominent generate action and a readable result panel. Keep the interface responsive so it works on both laptop and mobile screens. The final submission can replace the simple interface with a more polished card-based design if required.

9. Security, Privacy and Responsible Use

9.1 API Key Protection

The Gemini API key must remain on the server side. It should be loaded from an environment variable and excluded from Git tracking. Students should not place secrets inside HTML, JavaScript or a public GitHub repository.

9.2 Input Handling

The backend should validate request length and required fields. User input should not be used to execute code or commands. The application should return controlled errors without exposing internal configuration.

9.3 Educational Accuracy

Generative AI can produce incorrect or incomplete information. EduGenie therefore functions as an educational assistant. Important academic claims should be checked against trusted learning materials, teacher guidance or official references.

9.4 Privacy

The basic implementation does not need to permanently store student questions. Avoid collecting unnecessary personal information. If future versions add learning history, the project should clearly explain what is stored, why it is stored and how it is protected.

9.5 Responsible AI

The system should avoid presenting itself as a human teacher or as an authoritative institutional source. Generated content should be clearly understood as AI-assisted educational content.

10. Testing and Validation

10.1 Test Plan

Test ID	Test Case	Expected Result
T01	Valid question	Relevant explanatory response displayed
T02	Empty input	Validation message displayed
T03	Simplify a technical concept	Beginner-friendly explanation returned
T04	Generate quiz	Questions and answers returned
T05	Summarize text	Concise structured summary returned
T06	Recommendation request	Learning sequence returned
T07	Invalid task	Controlled error / fallback response
T08	Missing API key	Configuration error handled without exposing secret
T09	Long input	Request is handled or rejected safely
T10	Network/API failure	User receives a readable error message

10.2 Expected Results

A successful demonstration should show that the same application can perform multiple educational tasks through a common AI backend. The output should be readable, relevant to the supplied topic and organized according to the selected task.

10.3 Evaluation Criteria

Relevance of response; clarity of explanation; consistency of formatting; successful API communication; error handling; responsiveness of the interface; protection of the API credential; ease of adding new learning modules.

11. Execution and Deployment

11.1 Local Setup

```
# Create virtual environment
python -m venv venv

# Activate on Windows
venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Start the API server
uvicorn app:app --reload
```

11.2 Environment Setup

```
# .env
GEMINI_API_KEY=YOUR_KEY
```

11.3 Browser Access

After the server starts, open the local FastAPI address in a browser. The final application can serve the frontend directly or expose a static HTML page separately. For the academic demonstration, local execution is sufficient if the project requirements do not require cloud deployment.

11.4 GitHub Project Workflow

The project can be organized into the required development phases: Brainstorming → Requirement Analysis → Design → Planning → Development → Testing → Documentation → Demonstration. Each phase can be represented through folders, README sections, commits or project-management notes.

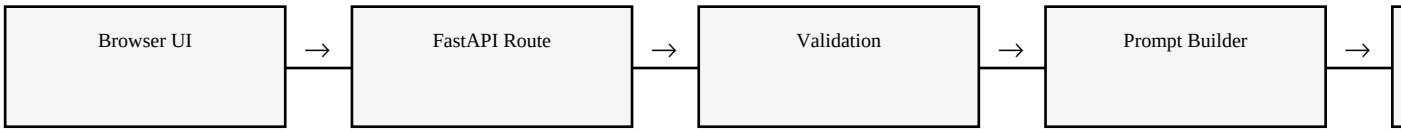
12. Detailed System Design

12.1 Request/Response Model

```
POST /generate

Request:
{
  "task": "simplify",
  "input_text": "Explain recursion in programming."
}

Response:
{
  "task": "simplify",
  "result": "..."
}
```

12.2 Component Interaction

Component Interaction



12.3 Sequence

Student → selects task → enters content → clicks Generate → browser sends JSON → FastAPI validates input → prompt builder creates task instructions → Gemini generates content → FastAPI returns JSON → browser displays the result.

12.4 Extensibility

Future modules can include flashcard generation, coding explanations, note-to-quiz conversion, multilingual explanations, difficulty adjustment, progress tracking and teacher-created topic packs. These are extension opportunities rather than required components of the base implementation.

13. Advantages, Limitations and Future Scope

13.1 Advantages

• One interface for multiple learning tasks.
• Faster access to explanations and revision material.
• Beginner-friendly output can reduce the barrier to difficult topics.
• Quiz generation supports active recall and self-testing.
• Modular FastAPI design supports future expansion.
• API credentials can be separated from source code.

13.2 Limitations

• AI-generated responses can contain factual errors.
• Output quality depends on the prompt and model response.
• Internet/API access is required for cloud-based Gemini generation.
• API usage may be subject to account limits and service availability.
• The basic prototype does not maintain a complete learner profile or verified curriculum database.

13.3 Future Scope

• Student login and personalized learning profiles.
• Learning history and progress dashboard.
• Adaptive difficulty based on quiz performance.
• PDF/notes upload and question generation from study material.
• Voice-based interaction.
• Multilingual learning assistance.
• Teacher dashboard for creating topic-specific learning activities.
• Retrieval-based grounding using institution-approved study material.

14. Project Development Phases

Phase	Activities	Expected Deliverable
Brainstorming	Identify student learning problems and AI opportunities	Project concept
Requirement Analysis	Define users, features and constraints	Requirements document
Design	Plan UI, API, prompt flow and architecture	Architecture + UI design
Planning	Divide tasks and define milestones	Team plan
Development	Implement frontend, FastAPI and Gemini integration	Working prototype
Testing	Run functional and error-handling tests	Test report
Documentation	Prepare report, README and screenshots	Project documentation
Demonstration	Record working application and explain workflow	Demo video

14.1 Suggested Team Roles

Role	Responsibility
Team Head / Project Lead	Coordination, integration and final submission
Backend Developer	FastAPI routes and Gemini integration
Frontend Developer	HTML/CSS/JavaScript interface
Testing & Documentation	Test cases, README, report and demo preparation
Research / Prompt Design	Prompt templates and feature validation

The same person may handle multiple roles in a small team. The table is a project-management suggestion rather than a mandatory staffing requirement.

15. Demonstration Script

15.1 Demo Scenario 1 — Question Answering

Enter: “What is object-oriented programming?” Select Ask a Question and generate. Demonstrate how the response explains the concept and provides a beginner-friendly structure.

15.2 Demo Scenario 2 — Simplification

Enter a difficult paragraph or technical concept. Select Simplify Concept. Show how the result converts complex wording into easier language.

15.3 Demo Scenario 3 — Quiz

Enter a topic such as Python functions. Select Generate Quiz. Show the generated questions and answers and explain that the quiz can be used for self-practice.

15.4 Demo Scenario 4 — Summary

Paste a short study passage. Select Summarize and demonstrate the key points returned by the model.

15.5 Demo Scenario 5 — Recommendation

Enter a topic such as “Learn Python from beginner to intermediate.” Select Study Recommendation and demonstrate the suggested sequence.

16. User Interface Documentation

16.1 Home Screen

The final project submission should include a screenshot of the EduGenie home page showing the project title, task selector, input area and Generate button.

SCREENSHOT PLACEHOLDER
Insert EduGenie Home Screen screenshot here
Recommended capture: full browser window showing the interface and selected task.

16.2 Result Screen

Insert a screenshot showing a generated explanation or quiz. The screenshot should clearly show both the user input/task and the returned AI response.

SCREENSHOT PLACEHOLDER
Insert generated-result screenshot here
Recommended capture: result area with readable output.

17. Code Appendix

17.1 requirements.txt

```
fastapi
uvicorn
python-dotenv
google-generativeai
jinja2
```

17.2 .gitignore

```
.env
venv/
__pycache__/
*.pyc
```

17.3 Example FastAPI Route

```
@app.post("/generate")
def generate(request: LearningRequest):
    if not request.input_text.strip():
        return {"error": "Please enter a learning topic or question."}

    prompt = build_prompt(
        request.task,
        request.input_text
    )

    response = model.generate_content(prompt)

    return {
        "task": request.task,
        "result": response.text
    }
```

17.4 Suggested README Contents

Project overview; features; technology stack; installation steps; environment setup; how to run; API endpoint; screenshots; testing; team members; GitHub phases; demo video link; limitations; future scope.

18. Conclusion

EduGenie demonstrates how Generative AI can be integrated into a practical educational application. By combining a simple web interface, a FastAPI backend and Google Gemini, the system provides a common platform for question answering, concept simplification, quiz generation, summarization and study recommendations.

The project is intentionally modular. The base prototype can be implemented with a small number of files while leaving room for more advanced capabilities such as personalization, retrieval from approved study materials and progress tracking. The most important consideration is that AI-generated learning content should be treated as assistance and reviewed when accuracy matters.

19. References

1. Project reference document supplied for the Generative AI track: EduGenie: Google Gemini Powered Learning Assistant. ■filecite■turn0file0■L2-L6■

2. Reference project model supplied by the user: LegalEase: AI Powered Legal Document Generator, Detailed Project Report. The structure and level of technical detail of this report were used as the formatting/content model. ■filecite■turn1file0■L1-L16■

3. Google Gemini / Generative AI documentation — consult the current Google documentation for the SDK and model configuration used by the team.

4. FastAPI documentation — for API development and request handling.

5. Python documentation — for project environment and application development.

20. Submission Checklist

Item	Status / Action
Project title	EduGenie: Google Gemini Powered Learning Assistant
Technology track	Generative AI with Google
Source code	Upload public GitHub repository
Project phases	Brainstorming → Requirement Analysis → Design → Planning → Development → Testing → Documentation → Demonstration
API key	Use environment variable; never publish the real key
Screenshots	Add actual application screenshots in the marked sections
Testing	Complete the test cases with actual observed results
Demo video	Upload to Google Drive with Anyone-with-the-link access if required
Final report	Export/submit this PDF after replacing placeholders and adding team details

Team Information

Field	Enter Details
Team Head	_____
Team Members	_____
College	_____
Course / Semester	_____
Academic Year	_____
GitHub Repository	_____
Demo Video	_____

End of Detailed Project Report